

MEMORIA TÉCNICA.....	2
IMPLEMENTACIÓN DE MÓDULO DE GESTIÓN DE INFORMES.....	2
1. Definición del Modelo de Datos Físico (Capa Lógica).....	2
2. Implementación de Consultas SQL (Extracción y Procesamiento).....	2
3. Interfaz de Usuario: Asistente de Generación (Wizard).....	2
4. Generación de Documentación Saliente (QWeb Reports).....	3
INFORME TÉCNICO: ASISTENTE DE GENERACIÓN DE INFORMES (WIZARD).....	4
1. Objetivo.....	4
2. Arquitectura del Modelo Transitorio (TransientModel).....	4
Campos de Control:.....	4
3. Lógica de Control y Validación.....	4
4. Interfaz de Usuario y Experiencia (UX).....	5
5. Configuración del Proyecto (Manifiesto).....	5
6. Conclusión.....	5
INFORME TÉCNICO: IMPLEMENTACIÓN DE VISTA SQL PARA INFORMES DE ALUMNOS.....	6
1. Objetivo.....	6
2. Definición del Modelo.....	6
Campos Definidos:.....	6
3. Lógica de la Consulta SQL.....	6
3.1. Uso de Alias y Uniones (JOINS).....	6
3.2. Gestión de Registros Relacionados.....	6
3.3. Funciones de Agregación y Agrupamiento.....	7
4. Seguridad y Acceso.....	7
5. Conclusión.....	7
INFORME TÉCNICO: IMPLEMENTACIÓN DE VISTA SQL PARA INFORMES DE CENTROS.....	8
1. Objetivo.....	8
2. Definición del Modelo.....	8
Campos Definidos (Identidad y Contacto):.....	8
Métricas de Volumen (Cálculos Agregados):.....	8
3. Lógica de la Consulta SQL.....	8
3.1. Pivotado sobre la Entidad Centro.....	8
3.2. Integración de Relaciones (JOINS).....	9
3.3. Funciones de Recuento y Agrupación.....	9
4. Seguridad y Acceso.....	9
5. Conclusión.....	9
INFORME TÉCNICO: CONFIGURACIÓN DE ACCIONES DE REPORTE (XML).....	10
1. Objetivo.....	10
2. Definición de Registros (ir.actions.report).....	10
3. Parámetros de Control y Seguridad.....	10
4. Conclusión.....	10

MEMORIA TÉCNICA

IMPLEMENTACIÓN DE MÓDULO DE GESTIÓN DE INFORMES

1. Definición del Modelo de Datos Físico (Capa Lógica)

Para la generación de informes con alta carga de datos y cálculos agregados, se optará por la creación de modelos de soporte basados en vistas de base de datos (`_auto = False`).

- **Modelo de Informe Individual (Alumno):** Este modelo actúa como receptor de una consulta SQL que integra la tabla de alumnos (`res.partner`), sus relaciones con centros y docentes (también en `res.partner`), y las entidades de control asistencia.gestion_asistencia y tareas.alumno. Define los campos necesarios para reflejar totales de horas acumuladas, conteo de tareas y registros de asistencia.
- **Modelo de Informe de Centro:** Orientado a la estructura organizativa, este modelo procesará la relación jerárquica entre el centro, su personal docente asignado y la nómina de alumnos matriculados.

2. Implementación de Consultas SQL (Extracción y Procesamiento)

La extracción de información se realizará mediante funciones `init()` que ejecutarán sentencias `CREATE OR REPLACE VIEW` en PostgreSQL.

- **Query de Alumno:** Utiliza una combinación de `LEFT JOIN` para asegurar la integridad de los datos aunque falten registros en alguna categoría (ej. alumnos sin tareas aún registradas). Se aplicarán funciones de agregado (`SUM`, `COUNT DISTINCT`) para obtener un **expediente histórico consolidado** del alumno, garantizando la integridad de los datos acumulados desde su alta en el sistema.
- **Query de Centro:** Se centrará en la agrupación de registros por identificador de centro, permitiendo consolidar en un único documento la relación de tutores y el volumen de alumnos bajo su supervisión, sin necesidad de filtros temporales estrictos si el objetivo es un listado de estado actual.

3. Interfaz de Usuario: Asistente de Generación (Wizard)

Con el fin de centralizar la gestión de informes y evitar la saturación de las vistas operativas, se implementará un modelo transitorio (`models.TransientModel`) accesible desde el menú "Gestión de Informes".

- **Comportamiento Dinámico:** El asistente presentará una interfaz simplificada donde el usuario selecciona la entidad (Alumno o Centro), eliminando la complejidad de

rangos temporales para ofrecer un **informe de estado actual y acumulado** inmediato.

- **Selección de Registros:** Se integrarán campos Many2one con filtros de búsqueda para seleccionar el alumno o centro específico sobre el cual se desea emitir el reporte, garantizando que el SQL reciba los identificadores exactos.

4. Generación de Documentación Saliente (QWeb Reports)

El proceso culminará en la creación de archivos PDF mediante el motor de renderizado QWeb.

- **Configuración de Acciones (Acciones IR):** Se han implementado registros en `ir.actions.report` que actúan como el nexo de unión entre los modelos SQL y las plantillas. Estas acciones están configuradas con `binding_model_id = False` para garantizar que el flujo de impresión sea exclusivo a través del Wizard, centralizando la lógica y protegiendo la interfaz de usuario de botones redundantes.
- **Plantillas QWeb:** Se diseñarán estructuras HTML semánticas que utilicen el `external_layout` de Odoo para mantener la coherencia institucional. La información se presentará utilizando los campos calculados de los modelos SQL de soporte.
- **Transformación de Datos:** La plantilla accederá a los registros pre-procesados por la vista SQL, permitiendo una representación visual profesional sin necesidad de realizar cálculos pesados en el momento de la impresión. Se aplicarán formatos de horas y fechas locales antes de la conversión final a PDF mediante `wkhtmltopdf`.

Para garantizar la **coherencia visual multiplataforma**, se ha procedido a la parametrización técnica del **Paper Format** en el sistema, ajustando el *Header Spacing* a **45mm**. Esta medida corrige los errores de renderizado de la librería *wkhtmltopdf* que provocaban el solapamiento de la cabecera institucional con el cuerpo de los datos en páginas sucesivas.

Consideraciones de Seguridad y Acceso: Cada uno de los nuevos componentes (modelos de vista y asistentes) contará con su correspondiente registro en el archivo `ir.model.access.csv`, asegurando que el personal administrativo y docente pueda acceder a las funciones de análisis según sus privilegios asignados.

INFORME TÉCNICO: ASISTENTE DE GENERACIÓN DE INFORMES (WIZARD)

1. Objetivo

El objetivo de este componente es proporcionar una interfaz transitoria (interfaz de tipo "ventana emergente") que permita al usuario parametrizar la salida del informe antes de su ejecución. El asistente actúa como un filtro dinámico que recolecta los criterios necesarios (entidad, fechas y selección específica) para alimentar de forma precisa a los modelos de vista SQL.

2. Arquitectura del Modelo Transitorio (TransientModel)

Se ha utilizado la clase `models.TransientModel` para definir el objeto `informe.gestion.wizard`. A diferencia de los modelos estándar, los registros de este modelo son temporales y se eliminan periódicamente de la base de datos, lo cual es ideal para procesos de configuración de reportes.

Añadir tabla: fichero py, nombre clase y función

Campos de Control:

- **x_tipo_informe:** Campo de selección (Selection) que determina el flujo de ejecución entre las opciones de "Alumno" o "Centro Docente".
- **x_alumno_id / x_centro_id:** Campos relacionales (Many2one) que vinculan el asistente con la tabla `res.partner`. Se han aplicado dominios (domain) específicos para asegurar que solo se muestren registros con los códigos técnicos correspondientes (`x_cod_alumno` o `x_cod_centro`).

3. Lógica de Control y Validación

La función principal `action_generar_pdf` gestiona la integridad de la petición mediante las siguientes acciones:

1. **Validación de Datos:** Antes de procesar la solicitud, el sistema verifica que los campos requeridos según el tipo de informe no estén vacíos, lanzando un error de usuario (`UserError`) en caso de omisión.
2. **Preparación del Diccionario de Datos:** Se construye un objeto `data` que encapsula los IDs de los registros seleccionados y los rangos de fechas.
3. **Llamada al Motor de Reportes:** Se ha optado por mantener la persistencia del asistente tras la impresión. Esta configuración permite al usuario administrativo realizar consultas iterativas (por ejemplo, generar el informe de varios alumnos uno

tras otro) sin perder el contexto de la búsqueda ni tener que reabrir el menú de informes, lo que agiliza significativamente el trabajo de secretaría ante volúmenes altos de registros.

4. Interfaz de Usuario y Experiencia (UX)

La vista XML (`informe_wizard_view.xml`) implementa una lógica de visibilidad dinámica mediante el atributo invisible:

- **Filtros Contextuales:** El campo de selección de **Alumno** solo es visible cuando la opción "Informe de Alumno" está activa. Del mismo modo, el campo de selección de **Centro Docente** solo aparece al marcar la opción "Informe de Centro".
- **Estructura de Pie de Página:** Se definen dos acciones claras: una acción principal (`btn-primary`) para disparar la lógica de Python y una acción de cancelación (`special="cancel"`) para cerrar la ventana sin procesar cambios.
- **Integración en Menú:** El asistente se integra en la estructura existente de `centro_docente`, creando una rama específica de "Gestión de Informes" para facilitar el acceso al usuario final.

5. Configuración del Proyecto (Manifiesto)

El archivo `__manifest__.py` asegura la cohesión del módulo mediante:

- **Declaración de Dependencias:** Se garantiza la carga previa de los módulos `alumnos`, `asistencia`, `centro_docente`, `personal_docente` y `tareas`, asegurando que los campos heredados en `res.partner` estén disponibles.
- **Carga de Datos:** Se establece el orden crítico de carga, situando la seguridad en primer lugar y las vistas del asistente antes que las plantillas de reporte para evitar errores de referencia circular.

6. Conclusión

Este enfoque de **expediente histórico** asegura que el reporte siempre refleje la totalidad del esfuerzo académico del alumno, eliminando errores de filtrado por periodos de inactividad.

INFORME TÉCNICO: IMPLEMENTACIÓN DE VISTA SQL PARA INFORMES DE ALUMNOS

1. Objetivo

El objetivo de este desarrollo es la creación de una capa de datos optimizada que consolide información dispersa en múltiples tablas (Alumnos, Centros, Docentes, Asistencias y Tareas) en una única entidad lógica de solo lectura. Esto permite que la generación de informes PDF sea eficiente, delegando el procesamiento de cálculos complejos al motor de la base de datos PostgreSQL.

2. Definición del Modelo

Se ha implementado el modelo `report.modelo.sql.alumno` utilizando la propiedad `_auto = False`. Esta configuración indica al ORM de Odoo que no debe crear una tabla física, sino que la estructura será definida manualmente mediante una vista SQL en el método `init()`.

Campos Definidos:

- **Identificadores:** `id` (vínculo único necesario para el ORM) y `x_cod_alumno`.
- **Datos Descriptivos:** `name`, `email`, `centro_name`, `centro_email` y `docente_name`.
- **Métricas Calculadas:** `x_horas_acumuladas`, `total_asistencias`, `total_horas_reales` y `total_tareas`.

3. Lógica de la Consulta SQL

La consulta se ha diseñado para garantizar la integridad de los datos incluso ante la ausencia de registros en las tablas relacionadas (Asistencias o Tareas) mediante el uso de uniones externas.

3.1. Uso de Alias y Uniones (JOINS)

Debido a que los datos de Alumnos, Centros y Docentes residen en la misma tabla física (`res_partner`), se han empleado alias diferenciados (`ALU`, `CEN`, `DOC`) para realizar las uniones mediante sus claves foráneas correspondientes (`x_centro_id` y `x_cod_docente_id`).

3.2. Gestión de Registros Relacionados

Se han utilizado `LEFT JOIN` hacia las tablas `asistencia_gestion_asistencia` y `tareas_alumno`. Esta elección es crítica para asegurar que, si un alumno no posee aún registros de asistencia o tareas, su información personal siga apareciendo en el informe en lugar de ser omitida por la consulta.

3.3. Funciones de Agregación y Agrupamiento

Para consolidar los datos en una única fila por alumno, se han aplicado las siguientes funciones:

- **COUNT(DISTINCT ID):** Para contabilizar de forma unívoca el número de asistencias y tareas, evitando el inflado de datos por producto cartesiano.
- **SUM:** Para totalizar las horas reales dedicadas a tareas.
- **COALESCE:** Para transformar valores nulos (NULL) en ceros (0), garantizando que el informe PDF no presente campos vacíos en las secciones numéricas.
- **GROUP BY:** La consulta agrupa por todos los campos descriptivos para permitir la correcta ejecución de las funciones de agregación sobre el ID del alumno.

4. Seguridad y Acceso

Se ha configurado una regla de acceso en el archivo `ir.model.access.csv` que otorga permisos de lectura (`perm_read`) al grupo de usuarios base. Al ser una vista de solo lectura, se han denegado explícitamente los permisos de escritura, creación y borrado, garantizando la inmutabilidad de los datos calculados desde la interfaz de usuario.

5. Conclusión

La estructura implementada proporciona una base sólida para el reporte. La separación de la lógica de datos (SQL) de la lógica de presentación (QWeb) facilita el mantenimiento del módulo y asegura un alto rendimiento, independientemente del volumen de registros de asistencia o tareas que existan en el sistema.

INFORME TÉCNICO: IMPLEMENTACIÓN DE VISTA SQL PARA INFORMES DE CENTROS

1. Objetivo

El objetivo de este desarrollo es proporcionar una entidad de datos centralizada que actúe como "maestra" para la gestión de Centros Docentes. La vista consolida la identidad del centro y genera métricas de volumen (conteo de alumnos y docentes) en una sola operación de lectura, optimizando el rendimiento al tratar a Odoo como un repositorio de estados consolidados.

2. Definición del Modelo

Se ha implementado el modelo `report.modelo.sql.centro` utilizando la propiedad `_auto = False`. Al ser una entidad de solo lectura, se alimenta directamente de una vista definida en PostgreSQL para evitar la carga de cálculos en el servidor de aplicaciones.

Campos Definidos (Identidad y Contacto):

- **Identificadores:** `id` (vínculo único necesario para el ORM) y `x_cod_centro` (Código técnico).
- **Datos Maestros:** `name` (Nombre del centro), `street` (Dirección física) y `city` (Ciudad).
- **Comunicación:** `phone` (Teléfono) y `email` (Correo electrónico institucional).

Métricas de Volumen (Cálculos Agregados):

- **total_alumnos:** Cálculo entero del número de alumnos vinculados al centro.
- **total_docentes:** Cálculo entero del personal docente asignado al centro.

3. Lógica de la Consulta SQL

La consulta se ha diseñado bajo un enfoque de **"Estado Actual" (Foto Fija)**. Este diseño simplifica el flujo de datos entre las distintas capas del ecosistema tecnológico (API Rest, Front-end y Base de Datos), evitando la necesidad de inyectar parámetros temporales en el repositorio interno de Odoo.

3.1. Pivotado sobre la Entidad Centro

La consulta utiliza la tabla `res_partner` (alias `CEN`) como eje central, filtrando estrictamente por los registros que poseen el código identificador de centro (`x_cod_centro IS NOT NULL`).

3.2. Integración de Relaciones (JOINS)

Se emplean dos LEFT JOIN estratégicos hacia la misma tabla física (res_partner) bajo los alias ALU (Alumnos) y DOC (Docentes):

- La vinculación se realiza mediante la clave foránea x_centro_id.
- Se asegura la presencia del centro en el informe aunque carezca de alumnos o docentes asignados en el momento de la consulta.

3.3. Funciones de Recuento y Agrupación

- **COUNT(DISTINCT ID):** Se utiliza para obtener el volumen real de participantes, eliminando duplicidades por producto cartesiano.
- **GROUP BY:** La consulta agrupa por todos los campos de identidad (name, street, city, phone, email), permitiendo una fila única por centro con sus métricas consolidadas.

4. Seguridad y Acceso

Se ha configurado una regla de acceso en el archivo ir.model.access.csv que otorga permisos de lectura (perm_read) al grupo de usuarios base. Al tratarse de una vista de solo lectura, se han denegado explícitamente los permisos de escritura, creación y borrado.

5. Conclusión

Este modelo completa la capa de datos del módulo de informes. La separación de la lógica de cabecera (SQL) y la lógica de listados detallados (ORM) garantiza un informe ligero y escalable, alineado con la función de Odoo como repositorio centralizado de la gestión académica.

INFORME TÉCNICO: CONFIGURACIÓN DE ACCIONES DE REPORTE (XML)

1. Objetivo

El objetivo de este componente es definir las "órdenes de ejecución" para el motor de informes de Odoo. Actúa como el puente técnico que recibe la instrucción del Wizard y la redirige hacia la plantilla visual correspondiente, asociando los datos del modelo SQL.

2. Definición de Registros (ir.actions.report)

Se han definido dos acciones principales en el archivo `report_actions.xml`:

- **action_report_alumno_sql**: Vinculada al modelo de vista SQL de alumnos.
- **action_report_centro_sql**: Vinculada al modelo de vista SQL de centros.

3. Parámetros de Control y Seguridad

Para asegurar un comportamiento profesional y controlado del módulo, se han aplicado los siguientes parámetros:

- **Nomenclatura Dinámica** (`print_report_name`): Se ha implementado una expresión Python que renombra el archivo PDF al descargarse utilizando el nombre del registro (ej. `Informe_Alumno_Juan_Perez.pdf`), facilitando la gestión documental externa.
- **Encapsulamiento de Interfaz**: Al desactivar el `binding_model_id`, se ocultan estas acciones de los menús contextuales de Odoo. Esto obliga a que toda generación de informes pase por el Wizard, asegurando que se cumplan las validaciones previas definidas en la capa lógica.
- **Motor de Renderizado**: Se especifica el uso de `qweb-pdf`, invocando a la librería `wkhtmltopdf` para garantizar una conversión fiel de HTML/CSS a documento impreso.

4. Conclusión

Este componente cierra el flujo de comunicación interna del módulo. Al desacoplar la acción de impresión del modelo original (`res.partner`) y vincularla a los modelos de soporte SQL, se garantiza que el reporte sea ligero y que la carga de datos esté optimizada antes de iniciar el proceso de diseño visual.